

Lab 2 - Encapsulation, Methods, Arrays, and Static Variables

Emily Antosch · Furkan Yildirim · Julian Moldenhauer
HAW Hamburg · TI

This lab introduces essential object-oriented programming concepts in Java through three progressive tasks. Students will learn encapsulation by implementing private variables with getter/setter methods, work with arrays and static variables for data management, and practice method design with validation. The exercises build from enhancing the Lab 1 name management system to creating grade calculators and bank account simulators, emphasizing proper class design, data protection, and modular programming principles.

Tasks

Task 1: Enhanced Name Management System	2
Task 2: Student Grade Calculator (Continuation of Lab 1, Task 4)	4
Task 3: Simple Bank Account Simulator	6
Task 4: Array Sorting and Searching	8
Task 5: Object Comparison and Equality	9
Task 6: Simple Inventory Management System	10
Lab Execution	12

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

Task 1: Enhanced Name Management System

Extend your Java program from Lab 1 to use proper object-oriented programming principles. This task builds on the Person class you created and adds encapsulation, class variables, and arrays.

Modify your name management program as follows:

- Make all instance variables in the Person class private
- Create getter and setter methods to access these variables
- Add validation in the setter for month (must be between 1-12)
- Replace the three separate person variables with an array of Person objects
- Add a static class variable to count the total number of Person objects
- Extend the menu to include option 5: display the total person count

Detailed requirements:

- The inputs in the individual input fields themselves do not need to be checked again for correctness, i.e., letters, numbers, special characters, meaning the inputs in the individual input fields must be meaningful and do not need to be checked by your program. However, all instance variables must now be protected from direct external access, i.e., they get the private attribute and access to them is done via so-called getter and setter methods. Only the month input needs to be exemplarily checked for plausibility.
- Introduce a class variable that counts the number of persons and outputs it on request, i.e., with the input of the number 5 in the selection menu, the number of persons is output. (This is still constant in this task, but could become significant in a later extension.)
- Furthermore, the individual persons are now to be stored in an array and no longer in separate variables, to enable an extension to a realistic number of persons.

Preparation

Again, first clarify the task by drawing the Person class according to UML notation and then creating a structure chart or flowchart that solves this task. Then define all necessary classes, methods, and variables in Java.

Assistance

The following code snippets demonstrate key concepts needed for this task:

Private variables and getter/setter methods:

```
1      public class Person {
2          private String firstName;
3          private String lastName;
4          private int day, month, year;
5          private static int personCount = 0;
6
7          // Constructor
8          public Person(String firstName, String lastName) {
9              this.firstName = firstName;
10             this.lastName = lastName;
11             personCount++; // Increment when new person is created
12         }
13
14         // Getter methods
15         public String getFirstName() {
16             return firstName;
17         }
18
19         // Setter with validation
20         public void setMonth(int month) {
```

Java

```
21         if (month ≥ 1 && month ≤ 12) {
22             this.month = month;
23         } else {
24             System.out.println("Invalid month! Must be between 1-12.");
25         }
26     }
27
28     // Static method to get person count
29     public static int getPersonCount() {
30         return personCount;
31     }
32 }
```

Using arrays instead of separate variables:

```
1  Person[] persons = new Person[3];
2  persons[0] = new Person("John", "Doe");
3  persons[1] = new Person("Jane", "Smith");
4  persons[2] = new Person("Bob", "Johnson");
```

Java

Task 2: Student Grade Calculator (Continuation of Lab 1, Task 4)

Preparation

Before implementing the solution, you must:

1. Design a class structure for the grade calculator system
2. Create a UML class diagram showing:
 - Class name (e.g., GradeCalculator)
 - All methods with their parameters and return types
 - Any necessary attributes/fields
3. Transfer your implementation into this class structure
4. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about the object-oriented design before writing code.

Task

This task continues the grade management system from Lab 1, Task 4. Create a Java program that manages student grades using arrays and methods. This task will help you practice working with arrays, methods, and calculations.

Write a program that:

- Stores grades for 5 students in an array
- Calculates the average grade
- Finds the highest and lowest grades
- Counts how many students passed (grade ≥ 60)
- Displays all results in a clear format

Your program should:

1. Ask the user to enter 5 student grades (0-100)
2. Store these grades in an array
3. Use separate methods for each calculation:
 - `int calculateAverage(int[] grades)` - returns the average
 - `int findHighest(int[] grades)` - returns the highest grade
 - `int findLowest(int[] grades)` - returns the lowest grade
 - `int countPassing(int[] grades)` - returns number of passing grades
 - `int[] deduplicatedGrades(int[] grades)` - returns a deduplicated array of occurred grades
4. Display all results with appropriate labels

Requirements

- Use an array to store the grades
- Create separate methods for each calculation
- Use loops to process the array
- Display results with clear formatting

Assistance

Array declaration and initialization:

```
1  int[] grades = new int[5];
2
3  // Reading grades into array
4  Scanner scanner = new Scanner(System.in);
5  for (int i = 0; i < grades.length; i++) {
6      System.out.print("Enter grade for student " + (i + 1) + ": ");
7      grades[i] = scanner.nextInt();
8  }
```

[Java](#)

Method example for calculating average:

```
1  public static double calculateAverage(int[] grades) {
2      int sum = 0;
```

[Java](#)

```
3      for (int grade : grades) {  
4          sum += grade;  
5      }  
6      return (double) sum / grades.length;  
7  }
```

Task 3: Simple Bank Account Simulator

Preparation

Before implementing the solution, you must:

1. Design a class structure for the bank account system
2. Create a UML class diagram showing:
 - Class name (BankAccount)
 - All private attributes (balance, accountHolder, etc.)
 - All public methods with their parameters and return types
 - Constructor(s)
3. Transfer your implementation into this class structure
4. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about encapsulation and class design before writing code.

Task

Create a simple bank account class that demonstrates encapsulation and method design. This task focuses on object-oriented programming principles.

Create a BankAccount class with the following features:

- Private balance variable
- Account holder name
- Methods for deposit, withdrawal, and balance inquiry
- Transaction validation (no negative deposits, sufficient funds for withdrawal)

Your program should:

1. Create a BankAccount object
2. Display a menu with options:
 - 1: Check balance
 - 2: Make deposit
 - 3: Make withdrawal
 - 4: Display account info
 - 0: Exit
3. Handle user input and call appropriate methods
4. Validate all transactions and display appropriate messages

Requirements

- Use private variables with public methods
- Validate all inputs (positive deposits, sufficient funds)
- Display clear success/error messages
- Use proper encapsulation principles

Assistance

Basic BankAccount class structure:

```
1      public class BankAccount {
2          private double balance;
3          private String accountHolder;
4
5          public BankAccount(String name, double initialBalance) {
6              this.accountHolder = name;
7              this.balance = initialBalance;
8          }
9
10         public boolean deposit(double amount) {
11             if (amount > 0) {
12                 balance += amount;
13                 return true;
14             }
15         }
```

Java

```
15         return false;
16     }
17
18     public boolean withdraw(double amount) {
19         if (amount > 0 && amount ≤ balance) {
20             balance -= amount;
21             return true;
22         }
23         return false;
24     }
25
26     public double getBalance() {
27         return balance;
28     }
29 }
```

Task 4: Array Sorting and Searching

Create a Java program that demonstrates array manipulation techniques including sorting and searching. This task will help you understand algorithm implementation and array processing.

- **Bubble sort**
 - Bubble Sort works by repeatedly stepping through the array, comparing adjacent elements and swapping them if they are in the wrong order, with larger elements “bubbling” to the end after each pass.
- **Linear Search**
 - Linear Search is a simple algorithm that examines each element in an array sequentially from the beginning until it finds the target value or reaches the end of the array, making it straightforward to implement but less efficient for large datasets.
- **Binary search**
 - Binary Search is an efficient algorithm that works on sorted arrays by repeatedly dividing the search space in half, comparing the target with the middle element to determine which half to search next. Both algorithms demonstrate different approaches to common programming problems with varying time complexities.

Write a program that:

- Creates an array of 10 random integers (1-100)
- Implements bubble sort to sort the array
- Implements linear search to find specific values
- Implements binary search (on sorted array)
- Displays arrays before/after sorting and search results

Your program should:

1. Generate and display an array of 10 random integers
2. Implement the following methods:
 - `void bubbleSort(int[] arr)` - sorts array using bubble sort
 - `int linearSearch(int[] arr, int target)` - returns index or -1
 - `int binarySearch(int[] arr, int target)` - returns index or -1 (requires sorted array)
 - `void printArray(int[] arr)` - displays array contents
3. Demonstrate all methods with user input for search values
4. Compare performance by counting comparisons in each search method

Requirements

- Use bubble sort algorithm for sorting
- Implement both linear and binary search
- Count and display number of comparisons for each search
- Handle cases where target value is not found

Assistance

Random array generation:

```
1  import java.util.Random;
2
3  Random random = new Random();
4  int[] numbers = new int[10];
5  for (int i = 0; i < numbers.length; i++) {
6      numbers[i] = random.nextInt(100) + 1; // 1-100
7  }
```

Java

Task 5: Object Comparison and Equality

Develop a comprehensive understanding of object comparison in Java by creating a Student class that implements proper equality checking and comparison methods.

Create a Student class with the following features:

- Private fields: studentID (int), name (String), grade (double)
- Override equals() and hashCode() methods
- Create methods to compare students by different criteria

Your program should:

1. Create an array of Student objects with sample data
2. Demonstrate equality checking between students
3. Sort students by grade (natural ordering)
4. Sort students by name using a custom comparator
5. Find duplicate students and display them
6. Search for students by ID and name

Requirements

- Override equals() correctly
- Create custom comparators for different sorting criteria
- Demonstrate all comparison methods with test data

Assistance

Example Student class with equals:

```
1      public class Student {
2          private int studentID;
3          private String name;
4          private double grade;
5
6          public Student(int studentID, String name, double grade) {
7              this.studentID = studentID;
8              this.name = name;
9              this.grade = grade;
10         }
11
12     }
```

[Java](#)

Task 6: Simple Inventory Management System

Preparation

Before implementing the solution, you must:

1. Design the complete class structure for the inventory management system
2. Create UML class diagrams for both classes showing:
 - **Item class:**
 - All private attributes (name, itemID, quantity, price, static variables)
 - All public methods with their parameters and return types
 - Constructor(s)
 - **Inventory class:**
 - All private attributes (items array, itemCount, etc.)
 - All public methods with their parameters and return types
 - Private helper methods (e.g., resizeArray)
 - Constructor(s)
3. Show the relationship between the two classes (composition/aggregation)
4. Transfer your implementation into this class structure
5. Only after completing the UML diagrams should you begin coding

This preparation step ensures you think about multi-class system design and relationships before writing code.

Task

Create a comprehensive inventory management system that combines all concepts learned in previous tasks. This capstone task integrates arrays, methods, encapsulation, and object-oriented design.

Develop an `Item` class and `Inventory` class to manage a collection of items with the following features:

- Item class with name, ID, quantity, and price
- Inventory class to manage multiple items
- Methods for adding, removing, updating, and searching items
- Generate reports and handle low stock warnings

Your program should:

1. Create an Inventory system with Item objects
2. Implement the following functionality:
 - Add new items to inventory
 - Update item quantities (restock/sell)
 - Search items by name or ID
 - Display low stock warnings (quantity < 5)
 - Calculate total inventory value
 - Generate inventory reports
3. Use arrays to store items and implement dynamic resizing
4. Provide a menu-driven interface for all operations

Requirements

- Use proper encapsulation with private variables
- Implement input validation for all operations
- Handle array resizing when adding new items
- Calculate and display inventory statistics
- Use static variables to track total number of items

Assistance

Item class structure:

```
1    public class Item {
2        private String name;
3        private int itemID;
4        private int quantity;
5        private double price;
6        private static int totalItems = 0;
7    }
```

Java

```
8      public Item(String name, int itemID, int quantity, double price) {
9          this.name = name;
10         this.itemID = itemID;
11         this.quantity = quantity;
12         this.price = price;
13         totalItems++;
14     }
15
16     public double getTotalValue() {
17         return quantity * price;
18     }
19
20     public boolean isLowStock() {
21         return quantity < 5;
22     }
23
24     // Getters and setters...
25 }
```

Inventory class with dynamic array:

```
1      public class Inventory {
2          private Item[] items;
3          private int itemCount;
4          private static final int INITIAL_CAPACITY = 10;
5
6          public Inventory() {
7              items = new Item[INITIAL_CAPACITY];
8              itemCount = 0;
9          }
10
11         public void addItem(Item item) {
12             if (itemCount ≥ items.length) {
13                 resizeArray();
14             }
15             items[itemCount++] = item;
16         }
17
18         private void resizeArray() {
19             Item[] newItems = new Item[items.length * 2];
20             System.arraycopy(items, 0, newItems, 0, itemCount);
21             items = newItems;
22         }
23
24         public Item findItemByID(int id) {
25             for (int i = 0; i < itemCount; i++) {
26                 if (items[i].getItemID() == id) {
27                     return items[i];
28                 }
29             }
30             return null;
31         }
32     }
```

Java

```
28         }  
29     }  
30     return null;  
31 }  
32 }
```

Lab Execution

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!